

Lab 4 — Code Hint Sheet

The Fashion VAE — a Latent Playground + Impostor Detector

You fill in **four** short pieces — two are one-line ideas that matter (the VAE loss and one KL number), one is a threshold plus a prediction, and one is a creative drawing. Everything else — the model, the encoder/decoder, the reparameterization trick, the training loop, and every plot — runs as-is. Type your answer in and **run the cell**; the built-in `assert` checks give instant feedback.

Step 2 — the VAE loss. A VAE trains on **two ideas at once**: `recon_loss` (rebuild the picture) and `kl_loss` (keep the latent cloud tidy near the origin). Both are already computed for you; your one line **combines** them, with the KL term weighted by `beta`.

Blank 1 — loss (combine the two terms).

- (a) `loss = recon_loss + beta * kl_loss` — **add** the KL penalty (scaled by `beta`) to the reconstruction. This is a VAE.
 - (b) `loss = recon_loss - beta * kl_loss` — *subtracting* the penalty **rewards** a messy latent; the space stops being sampleable. Wrong sign.
 - (c) `loss = recon_loss * beta * kl_loss` — *multiplying* is not the recipe; when `kl_loss` is near 0 the whole loss collapses to ≈ 0 . Wrong.
- (a) is the VAE objective (reconstruction + $\beta \cdot \text{KL}$). The self-check asserts `loss == recon_loss + BETA*kl_loss` (scalar, finite, $\text{KL} \geq 0$), so both the wrong sign (b) and the wrong operator (c) **fail** the assert with a clear message.

Step 3 — KL by hand. No code — a calculator. For one latent cloud with `mu = [1,0]`, `logvar = [0,0]`, plug into $\text{KL} = -\frac{1}{2} \sum_i (1 + \log \text{var}_i - \mu_i^2 - e^{\log \text{var}_i})$ with $e^0 = 1$: dim 1 gives $1 + 0 - 1 - 1 = -1$, dim 2 gives $1 + 0 - 0 - 1 = 0$, so the sum is -1 and $\text{KL} = -\frac{1}{2}(-1)$.

Blank 2 — kl_by_hand (one number).

- (a) `kl_by_hand = 0.5` — $-\frac{1}{2} \times (-1) = 0.5$. Matches the formula the code evaluates on the same numbers.
 - (b) `kl_by_hand = -0.5` — this is the *sum* times $\frac{1}{2}$ *without* the leading minus sign; the $-\frac{1}{2}$ flips it to $+0.5$. Fails the check.
 - (c) `kl_by_hand = 1.0` — dropped the factor $\frac{1}{2}$. Fails the check.
- (a) is exact. The `assert` recomputes the same KL formula in code and demands your number match to 10^{-6} , so recompute carefully rather than guessing.

Step 8 — the impostor threshold (+ a prediction). Reconstruction error is the impostor score: clothes rebuild well (low), unseen digits rebuild badly (high). The rule: put the line at the **90th percentile of the *normal* (clothes) errors**, so about **10%** of real clothes are false alarms.

Blank 3a — THRESHOLD.

- (a) `THRESHOLD = np.percentile(normal_errors, 90)` — the 90th percentile of the **normal** errors, exactly the rule.
- (b) `THRESHOLD = np.percentile(impostor_errors, 90)` — wrong array: you would calibrate on the *digits* you are trying to catch, not the clothes.
- (c) `THRESHOLD = normal_errors.mean()` — the mean sits far below the 90th percentile, so roughly half of all clothes would false-alarm.

Blank 3b — expected_false_alarms (predict, then check).

- (a) `expected_false_alarms = round(0.10 * len(normal_errors))` — by construction the 90th percentile leaves **10%** of normals above the line.
 - (b) `expected_false_alarms = round(0.90 * len(normal_errors))` — confuses the percentile (90) with the false-alarm rate (10%).
 - (c) `expected_false_alarms = 0` — a 90th-percentile cut always flags $\approx 10\%$ of normals, never 0.
- (a) for both. The prediction `assert` confirms the count you predicted is what the threshold actually produces.

Step 9 — draw your own impostor. Edit the 16×16 ASCII grid: `'#'` is ink, `'.'` is blank. The rule is only about **format** — keep **every row exactly 16 characters** and use only `'#'` and `'.'`. The code upsamples it to 28×28 and tells you whether the VAE flags it as an impostor.

Blank 4 — DOODLE (the drawing).

- (a) Any shape — the shipped **spiral**, your initials, a squiggle, a molecule sketch — as long as it is a list of **16 rows**, each exactly **16 chars** of only `'#'`/`'.'`. This passes and gets scored against the threshold.
- (b) Rows of *unequal* length, a wrong number of rows, or a stray character like `'x'` or a space — trips the grid sanity-check `assert`.
- (c) An empty grid (all `'.'`) — runs, but there is nothing for the detector to judge; draw *something* thin (lines and text flag easily; a big solid blob may slip through).

*Answers self-check in the notebook: the loss is a finite scalar with the KL term non-negative, **kl_by_hand** matches the code's KL, the threshold flags about 10% of clothes and separates digits well ($AUROC > 0.85$), and your doodle parses to (1, 784). The payoff is the tidy VAE latent map (vs. the scattered plain-AE one), the sneaker→boot morph, generated clothes, and your own doodle judged by the detector.*